Abstraction Project Notes

Brandon Tang

Contents

1. Symbolic Simulation for Functional Properties

The input to symbolic trajectory evaluation consists of an antecedent and a list of output constraints.

- The antecedent maps BDD variables to input wires in the circuit
- The output constraints map BDD expressions to signals in the circuit
  - These allow us to assert if a certain wires should be high/low/either at a certain time step for given conditions based on the antecedent

These were traditionally written in the form of LTL formulae, but have been rewritten into a 4 tuple form in VOSSII as follows:

$$[(\text{signal}, \text{tickStart} : \text{tickEnd}, \text{highExpr}, \text{lowExpr})]$$

This is equivalent to the LTL formula `N^tick ((highExpr -> (signal is 1) and lowExpr -> (signal is 0))) for each tick in the range`.

For an antecedent, this means that we will set the signal to be true if the `highExpr` is true and set the signal to be false if the `lowExpr` is true. Otherwise the signal is `X`.

For output constraints this means that when the assignment of BDD variables make `highExpr` true, we need the signal to be high from `tickStart` to `tickEnd`. Similarly for `lowExpr`. Thus each output constraint tuple can be split into 2 parts: the positive constraint where we need the signal to be true under a certain condition, and the negative constraint where we need the signal to be false under a certain condition. We let the `highExpr` and `lowExpr` be termed as the guards of the constraint.

Jaspergold fully implements symbolic simulation without support for abstraction with an indexing relation. Here, antecedents are not dual rail but rather just singular expressions, this means that we have no symbolic indexing and abstraction beyond the inital Xs present in the circuit. The output constraints here also do not have guards. To conditionally check properties, we would adjust the input constraint `cin`.

For the purposes of doing symbolic indexing, we settled on using the 4-tuple form of antecedents and output. This makes it easier to port over the existing theory on indexing transformations from the STE papers. Furthermore, Jasper's symbolic simulation engine accepts stimuli in the form of the 4-tuple antecedent.

2. Indexing Relations

Indexing relations are used to represent abstractions of circuit inputs. This means that they merge sets of circuit inputs into cases.

Indexing relations $R[X, C, T]$ are boolean formulae over BDD variables where

- X are the indexing BDD variables that perform the symbolic indexing
- T are the target BDD variables that are indexed. These correspond to input wires in the circuit
- C are the symbolic constants that correspond to target wires that should not be indexed

The indexing relation can be interpreted as follows:

- For each X, C, we cover the cases T, C where $\exists T R[X, C, T]$.
- Since the indexing relation can be a many to many relation, we can have multiple T, C cases covered by a single X, C case indexing and vice versa.

Note that the indexing relation $R'[(X, C'), \emptyset, (C, T)] = R[X, C, T] \wedge \bigwedge(C = C')$ that doesn't use symbolic constants is equivalent to $R[X, C, T]$ from the perspective of doing symbolic simulation and its associated operations (see below). The use of symbolic constants does not improve the expressive power of our indexing relations, but rather helps to simplify them in order to make them more efficient to use.

2.1. Preimage Operations

We define some operations involving the indexing relation here:

Suppose
- $P[C, T]$ represents some set of input cases (e.g. (C, T) is in the set iff P[C, T] is true)
- $R[X, C, T]$ is an indexing relation

The **weak preimage** of P under R,

$$P_R[X, C] = \exists T(R[X, C, T] \wedge P[C, T])$$

is the set of (X, C) that cover at least one (T, C) case in P

The **strong preimage** of P under R,

$$P^R[X, C] = P_R \wedge \neg\exists T(R[X, C, T] \wedge \neg P[C, T])$$

is the set of (X, C) that cover at least one case in P and don't cover any cases not in P.

2.2. Image Operation

Taking the image of $H[X, C]$ returns the cases that are indexed by $H[X, C]$ under R.

$$\mathrm{im}(H, R)[C, T] = \exists X(R[X, C, T] \wedge H[X, C])$$

While this is not directly used for finding proofs, it is useful for extracting counter examples for disproven properties.

2.3. Relationship between Preimages

For any fixed R and any given P and (X, C) we have one of the following cases:
- X, C indexes cases only in P
- X, C indexes cases only in $\neg P$
- X, C indexes cases in both P and $\neg P$
- X, C indexes cases in neither P nor $\neg P$

Any X, C that fall into the 4th category must correspond to empty indexing cases, i.e. don't map onto any (C, T) at all. Notice that this is independent of P, thus, we term the union of the first 3 cases as the domain of R.

$$\mathrm{dom}(R)[X, C] = \exists T(R[X, C, T])$$

Observe that
- The weak preimage is the set of X, C that fall into the 1st and 3rd categories.
- The strong preimage is the set of X, C that fall into the 1st category.

We can note that the following are then also equivalent definitions of the strong preimage operation:

$$\begin{aligned} P^R[X, C] &= \mathrm{dom}(R) \wedge \overline{\overline{P}_R} \\ &= \mathrm{dom}(R) \wedge (\forall T(R[X, C, T] \to P[C, T])) \end{aligned}$$

2.4. Indexing Coverage Condition

For each property that we wish to check, we let the high and low expressions be P_i and Q_i respectively. The total space of inputs we need to check is then $B[C, T] = \bigvee_i (P_i \vee Q_i)$. In order for our abstraction to properly cover the space of required inputs, we require that

$$\forall T \forall C(B[T, C] \to \exists X(R[X, C, T]))$$

Generally we will be having guards that are equivalent to true, i.e. should hold for all inputs. In this case, $B[C,T] \equiv \text{True}$ so we coverage condition takes the simpler form:

$$\forall T \forall C \exists X (R[X,C,T])$$

3. Indexing Transformation and Symbolic Simulation Procedure

Suppose we are given an indexing relation, `idx_rel`, an antecedent list and an output constraint list. We will describe the procedure to prove that the output constraints hold.

We first apply the strong preimge operations on the high and low expressions of the antecedent tuples as follows:

```
antv = [(signal, tickStart:tickEnd, strongPreimage idx_rel highExpr, strongPreimage idx_rel lowExpr)]
```

Next we run the symbolic simulator with the transformed antecedent. This produces an evaluation sequence which will contain `sigHighExpr` and `sigLowExpr` BDD expressions for when a signal at a certain time step will definitely be high or low. If for a given assignment, neither of these expressions are true, then the signal is unknown. For a given signal and time step s, we let the high expression be H_s and the low expression be L_s.[1]

From this evaluation sequence, we can then check the output constraints. Suppose want to check a output constraint of the form `(signal, tickStart:tickEnd, P, Q)` where $P[C,T]$ and $Q[C,T]$ are the BDD expressions that correspond to inputs on which a signal should be high and low respectively. We will transform the output constraint to `(signal, tickStart:tickEnd, P_R, Q_R)` where P_R and Q_R are the weak preimages of P and Q under the indexing relation.

Then we analyse the `highexpr` and `lowexpr` BDD expressions from the evaluation sequence for the signal at each of the ticks in the tick range. Suppose that at a given tick t, for the property wire we have some the high expression as $H[X,C]$ and the low expression as $L[X,C]$. We term the high expression of the signal as the "residual".

We will check if $(P_R \rightarrow H)[X,C] \equiv \text{True}$ and whether $(Q_R \rightarrow L)[X,C] \equiv \text{True}$. If both of these hold, then the output constraint is satisfied.

If we have $(P^R \wedge L) \neq \text{False}$ it means that some indexing cases that only index into P actually cause s to be low, this is a counter example to the positive part of property so the positive part of property is disproven. We can inspect the cases indexed as $\text{im}(P^R \wedge L, R)$ to get the actual counter example in terms of the original circuit inputs.

Similarly, if we have $(Q^R \wedge H) \neq \text{False}$, we have disproven the negative part of the property.

It is possible to not prove the property but also have no counter examples. This can happen in cases where the antecedent does not provide enough information to prove or disprove the property. This is called a weak disagreement and requires changing the indexing relation to obtain a proof/counter example.

4. Correctness of Indexing Transformation and Output Constraint Checking

We prove that the above procedure for indexing transformation and output constraint checking is correct.

[1] We will sometimes drop the subscript when there is only 1 relevant signal and time step being considered.

Here we only need to focus on proving the correctness of positive output constraint checking, i.e. that under a certain condition, a signal will be high. The proof for the negative output constraint checking is analagous.

With our formulation of relational properties later on, we don't even need to deal with guards, but our proof of correctness will deal with the general case since guards can help with environmental constraints (discussed later).

4.1. Symbolic Simulation Invariant

Consider some residual $H_s[X, C]$ in a signal $s(C, T)$ at a fixed time step. We always have the fact that

$$H_s[X, C] \wedge R[X, C, T] \rightarrow s(C, T)$$

(For all X, C, T)

This can be proved via induction on the fan-in of S.

The basecase is where s is an input or state variable corresponding to a certain time step, with `highexpr` P. In this case, since we apply the strong preimage to the antecedents, we have that

$$H = P^R = \text{dom}(R) \wedge \forall T(R[X, C, T] \rightarrow P[C, T])$$

This directly gives us that for all X, C, T where $H[X, C] \wedge R[X, C, T]$, we have $P[C, T]$. But $s(C, T) = P[C, T]$ so we are done.[2]

An inductive case example:

Suppose $s = s_1 \wedge s_2$. We have Q_1 and Q_2 as the residuals of s_1 and s_2 respectively.

- $Q_1[X, C] \wedge R[X, C, T] \rightarrow s_1(C, T)$
- $Q_2[X, C] \wedge R[X, C, T] \rightarrow s_2(C, T)$
- So $Q_1[X, C] \wedge Q_2[X, C] \wedge R[X, C, T] \rightarrow s_1 \wedge s_2$
- But is equivalent to $H[X, C] \wedge R[X, C, T] \rightarrow s$ since $Q = Q_1 \wedge Q_2$ via the symbolic simulator

4.2. Output Constraint Checking

Output constraints that we wish to prove are of the form:

$$\forall C \forall T(P[T, C] \rightarrow s(C, T))$$

If we have that $P_R[X, C] \rightarrow H[X, C]$ and the coverage condition $\forall T \forall C(B[C, T] \rightarrow \exists X(R[X, C, T]))$ holds, then this will be true.

First remember that $P_R = \exists T(R[X, C, T] \wedge P[C, T])$.

Fix some C, T such that $P[C, T]$ is true.

- Since $P[C, T]$ is true, then $\forall X(R[X, C, T] \rightarrow P_R[X, C])$ by the definition of P_R.
- Since $P[C, T]$ is true, so is $B[C, T]$
- Consider some X^* such that $R[X^*, C, T]$ is true, this must exist by the coverage condition
- Since $P[T, C]$ and $R[X^*, C, T]$ are true, then we must have $P_R[X^*, C]$
- But now since $P_R[X, C] \rightarrow H[X, C]$, we have $H[X^*, C]$ as well
- Using the symbolic simulation invariant, since $H[X^*, C]$ and $R[X^*, C, T]$ are true, then $s(C, T)$ is true
- Since this did not rely on the values of C, T, then we have that $\forall C \forall T(P[T, C] \rightarrow s(C, T))$ is true

[2]Note that we don't use the domain part for this. Indeed, the analysis will still work, but this serves to remove useless/inconsistent indexing cases. This is useful for counterexample analysis later on.

4.2.1. Alternative Checking by Reversing the Indexing

Another way to do the check is to take the image of the residual under the indexing relation, i.e. $\mathrm{im}(H, R)[C, T]$. This will symbolically represent all the cases for which we know the property will hold.

$$\begin{aligned} C, T \in \mathrm{im}(H, R) &\Rightarrow \exists X(H[X, C] \wedge R[X, C, T]) \\ &\Rightarrow H[X^*, C] \wedge R[X^*, C, T] \text{ for some } X^* \\ &\Rightarrow s(C, T) \text{ by symbolic simulation invariant} \end{aligned}$$

We can then check that $P \rightarrow \mathrm{im}(H, R) \equiv \mathrm{True}$. If so then the property is true. On some circuits and indexing relations, this method can avoid weak disagreements compared to the above method. This is particularly true if some cases in P are indexed by multiple X, C. However, if the number of indexing variables is much less than the number of target variables, this method can be result in large BDDs that may be infeasible to compute.

4.3. Negative Output Constraints

We can set up an analagous invariant for the `lowexpr` values in the symbolic simulation. We will then be able to prove that if L is the lowexpr of $s(C, T)$, then $(L[X, C] \wedge R[X, C, T] \rightarrow \neg s(C, T))$.

We can then prove output constraints of the form $\forall C \forall T(P[T, C] \rightarrow \neg s(C, T))$ in a similar manner to the highexpr case.

For our purposes where we use just need to prove that a certain wire is always high, we don't need this component. That being said, analysis of $L[X, C]$ is important for finding weak disagreements and counter examples.

4.4. Counter Example Analysis

Suppose that we have $P^R \wedge L \neq \mathrm{False}$. Let (X, C) be such that $(P^R \wedge L)[X, C]$ is true. We show that (X, C) is a counter example to the property $\forall C \forall T(P(C, T) \rightarrow s(C, T))$.

We select some T^* such that $R[X, C, T^*]$ is true. This must exist since $P^R = \mathrm{dom}(R) \wedge \forall T(R[X, C, T] \rightarrow P[C, T])$ so (X, C) is in the domain of R, meaning that $\exists T R[X, C, T]$.

Now since $\forall T(R[X, C, T] \rightarrow P[C, T])$, we must also have that $P[C, T^*]$ is true.

The symbolic simulation invariant for negative properties says that $L[X, C] \wedge R[X, C, T] \rightarrow \neg s(C, T)$. Since $L[X, C]$ and $R[X, C, T^*]$ are true, then $\neg s(C, T^*)$ is true.

By considering the example (C, T^*), we have $\exists C \exists T(P(C, T) \wedge \neg s(C, T)) \equiv \neg \forall C \forall T(P(C, T) \rightarrow s(C, T))$ being true, so the property is disproven.

It is practical to note that that the set of counter examples we find is $\{(C, T) \mid \exists X(R[X, C, T] \wedge L[X, C] \wedge P^R[X, C])\}$ which is described by the image operation on $P^R \wedge L$, $\mathrm{im}(P^R \wedge L, R)$.

We can prove that the counter example analysis for negative properties is correct in a similar manner.

5. Converting Relational Constraints to Functional Constraints

Traditionally STE only deals with functional properties. However, we can employ a trick to also check relational properties.

Our relational properties are initially written as SytemVerilog assertions. When these assertions are loaded into Jasper Gold, they can be treated as pseudo-signals. Jasper Gold internally treats these properties as additional circuitary that is added onto the specification circuit and has an output signal

that corresponds to the correctness of the property at each time.[3]

This means that for us, we can treat these properties as functional properties that just correspond to checking if the pseudo-wire is true. We call this pseudo-wire as the "**property wire**".

Properties of this form are actually functional properties and thus can be written in the above 4 tupe form easily as `cout = [(property_wire, k:k, TRUE, FALSE)]`. We can prove this for k and then use the time shift to prove the rest of of the ticks > k. We choose k such that k is the first tick in which we can prove the property to be true.

5.1. Indexing Transformations for Relational Properties

An additional benefit from this encoding method is that the property guards are extremely simple and thus have some nice properties related to the indexing transformation.

To check the properties, we would usually apply the weak preimage image operation on the guards. However, we have the following:

- $\mathrm{True}_R = \mathrm{True}^R = \mathrm{dom}(R)[X, C]$
- $\mathrm{False}_R = \mathrm{False}^R = \mathrm{False}$

This means that we only need to take 1 single preimage operation to get the domain and use that for checking all the various properties written in this form. In fact, computing the domain of an indexing relation generated from our automatic abstraction algorithm is a very simple operation that will be described later.

Furthermore, analysis of counter examples is also simplified.

$$\mathrm{True}^R \wedge L = \mathrm{dom}(R)[X, C] \wedge L = L$$

The 2nd equality comes from the fact that we take the strong preimage of the antecedents, so for any signal and time step s, H_s and L_s do not include any cases that are not in the domain of R. I.e. $H_s \vee L_s \rightarrow \mathrm{dom}(R)$. This can be proven with induction over the circuit in the same manner as the symbolic simulation invariants.

So any time L is not false, we have found a counter example. Specifically, any T, C in $\mathrm{im}(L, R)$ is a counter example.

5.2. Simplifying Properties for Automatic Abstraction

While we don't need any special form of properties with a manually crafted indexing relation, doing so can be very helpful for doing automatic abstraction. While the traditional way of doing abstraction works by forming the abstractions on all outputs of the specification circuit, we can sometimes derive better results by performing the abstraction algorithm directly from the wire that represents the property.

To assist with this process, we observe that each SV assertion can have its logic be shifted out of the property and into the surrounding specification circuit.

This means that we only need to check properties that are of the form `##k signal_name` where k is the number of clock cycles required to establish a certain property and `signal_name` is the name of the signal that corresponds to the property being true.

Since this signal is a real signal is jasper gold (as compared to the property wire), we can then run the automatic abstraction algorithm directly from this wire to find the indexing relation that will cover all the cases where the property wire is true.

[3] We can view the simulated values of these pseudo-wires with `check_symsim -sequence $eval_seq -get $assertions -verbose`.

6. Automatic Abstraction

The original automatic abstraction algorithm from 2007 performed a recursive back propagation from the functional output of the specification to automatically create an indexing relation. By calling `bp(C, specOutput, x_0, not x_0, name)`, we would produce an abstraction that considered all the cases that caused the `specOutput` to be `true` or `false`, while keeping all the signals in `C` as symbolic constants.

The cost savings of the indexing relation came from only using 1 BDD variable on `XNOR` gates and doing a binary encoding on AND gates with ≥ 3 inputs.

- For the XNOR gate, we only needed to consider if it was true or false, and it was true by having both inputs equal and false by having both inputs different.
- For the AND gate with n inputs, we have $n + 1$ cases. Either any of the inputs are false, i.e. the first n cases, or all of them are true, the last case. This would be constrasted with the 2^n cases that would be required if we did not use the indexing relation.

This has been reimplemented and improved.

6.1. Application onto Circuits or Specifications

The traditional way of applying the automatic abstraction algorithm is to run it on the output of the specification circuit. This should produce an indexing relation that covers all cases that cause the output of the specification to either be true or false.

6.2. Application onto Property Wires

Another way we can use this is to specifically look at all possible cases that are required for a property to be true and ensure that we abstract over them in a manner that we can prove the property being true. This leads us to running the automatic abstraction algorithm on the property wire itself.

Since we are looking specifically at properties of the form "the property wire is true", we don't need to be concerned with considering any indexing case where the property wire is false (because there shouldn't be any). This means that we should run

```
bp(C, property_wire, TRUE, FALSE, name)
```

To generate the required indexing relation.

For this to work, the property wire must be (represented by) a real Jasper Gold wire, which can be done by shifting the logic for the property wire out of the property and into the surrounding circuit as described above.

7. Partitioned Indexing Relations

An important subclass of indexing relations that we consider is the partitioned indexing relation. A partitioned indexing relation is one that can be expressed in the following form:

```
[(expr = targVar / Cexpr, hexpr, lexpr)]
```

This is a representation of the indexing relation $\bigwedge(\text{hexpr} \rightarrow \text{expr} \wedge \text{lexpr} \rightarrow \overline{\text{expr}})$ where `expr` is either some **target variable** or an **expression of symbolic constants** while `hexpr` and `lexpr` are in terms of only the indexing variables.

The automatic abstraction algorithm produces partitioned abstraction indexing relations. The manually constructed indexing relations that we use in our examples are also partitioned abstractions.

This representation allows more efficient computation of preimages which are critical for allowing us to perform indexing transformations at scale.

7.1. Efficient Weak Preimage Computation

We first normalise R into

$$R = S[X, C] \wedge U[X, T]$$

where $U[X, T] = \bigwedge_{t_i \in \text{ TargVars}} (h_i \to t_i \wedge l_i \to \overline{t_i})$

We can then make several observations.

Firstly,

$$\text{dom}(R)[X, C] = S \wedge \bigwedge_i \overline{h_i \wedge l_i}$$

Proof:

$$\begin{aligned}
\text{dom}(R)[X, C] &= \exists T R[X, C, T] \\
&= S[X, C] \wedge \exists T U[X, T] \\
&= S[X, C] \wedge \bigwedge_i (\exists t_i (h_i \to t_i \wedge l_i \to \overline{t_i})) \\
&= S[X, C] \wedge \bigwedge_i (((h_i \to 1) \wedge (l_i \to 0)) \vee ((h_i \to 0 \wedge l_i \to 1))) \\
&= S[X, C] \wedge \bigwedge_I (\neg l_i \vee \neg h_i) \\
&= S \wedge \bigwedge_i \overline{h_i \wedge l_i}
\end{aligned}$$

Thus we are able to compute the domain of the indexing relation easily.

Secondly, to compute the preimage of some guard $P[C, T]$, we let $R \downarrow P$ denote that the part of the indexing relation that mentions target variables present in P. Specifically, if $\mathcal{F} = \text{FreeTargVars}(P)$ then

$$R \downarrow P = S[X, C] \wedge \bigwedge_{t_i \in \mathcal{F}} (h_i \to t_i \wedge l_i \to \overline{t_i})$$

We have that $P_R = \text{dom}(R) \wedge P_{R \downarrow P}$

Proof:

$$
\begin{aligned}
P_{R[X,C]} &= \exists T(R[X,C,T] \wedge P[C,T]) \\
&= \exists T\left(S[X,C] \wedge \bigwedge_{t_i \notin \mathcal{F}} (h_i \to t_i \wedge l_i \to \overline{t_i}) \wedge \bigwedge_{t_i \in \mathcal{F}} (h_i \to t_i \wedge l_i \to \overline{t_i}) \wedge P[C,T]\right) \\
&= S[X,C] \wedge \exists T_{\neg\mathcal{F}}\left(\bigwedge_{t_i \notin \mathcal{F}} (h_i \to t_i \wedge l_i \to \overline{t_i})\right) \\
&\quad \wedge \exists T_{\mathcal{F}}\left(\bigwedge_{t_i \in \mathcal{F}} (h_i \to t_i \wedge l_i \to \overline{t_i}) \wedge P[C,T]\right) \\
&= S[X,C] \wedge \exists T_{\neg\mathcal{F}}\left(\bigwedge_{t_i \notin \mathcal{F}} (h_i \to t_i \wedge l_i \to \overline{t_i})\right) \wedge \exists T_{\mathcal{F}}\left(\bigwedge_{t_i \in \mathcal{F}} (h_i \to t_i \wedge l_i \to \overline{t_i})\right) \\
&\quad \wedge S[X,C] \wedge \exists T_{\mathcal{F}}\left(\bigwedge_{t_i \in \mathcal{F}} (h_i \to t_i \wedge l_i \to \overline{t_i}) \wedge P[C,T]\right) \\
&= S[X,C] \wedge \exists T\left(\bigwedge_{t_i \in \text{ targVars}} (h_i \to t_i \wedge l_i \to \overline{t_i})\right) \\
&\quad \wedge \exists T_{\mathcal{F}}\left(S[X,C] \wedge \bigwedge_{t_i \in \mathcal{F}} (h_i \to t_i \wedge l_i \to \overline{t_i}) \wedge P[C,T]\right) \\
&= S[X,C] \wedge \bigwedge_i \left(\overline{h_i \wedge l_i}\right) \wedge P_{R\downarrow P} \\
&= \text{dom}(R) \wedge P_{R\downarrow P}
\end{aligned}
$$

Now, we can even further consider common cases of P that we will be constructing preimages for.

$$
P_R = \begin{cases} \text{dom}(R) \wedge P \text{ if } \ \mathcal{F} \cap \text{TargVars} = \emptyset \\ \text{dom}(R) \wedge \overline{l_i} \text{ if } \ P = t_i \\ \text{dom}(R) \wedge \overline{h_i} \text{ if } \ P = \overline{t_i} \\ \text{dom}(R) \wedge P_{R\downarrow P} \text{ otherwise} \end{cases}
$$

Proof:

If $\mathcal{F} \cap \text{TargVars} = \emptyset$, $R \downarrow P = \text{True}$ so

$$
P_R = \text{dom}(R) \wedge \exists T(\text{True} \wedge P[C,T]) = \text{dom}(R) \wedge P
$$

If $P = t_i$, $R \downarrow P = (h_i \to t_i) \wedge (l_i \to \overline{t_i})$, so

$$
P_R = \text{dom}(R) \wedge \exists t_i((h_i \to t_i) \wedge (l_i \to \overline{t_i}) \wedge t_i) = \text{dom}(R) \wedge \overline{l_i}
$$

7.2. Efficient Strong Preimage Computation

Observe that

$$
\begin{aligned}
P^R &= \text{dom}(R) \wedge \overline{\overline{P}_R} \\
&= \text{dom}(R) \wedge \neg\left(\text{dom}(R) \wedge P_{R\downarrow\overline{P}}\right) \\
&= \text{dom}(R) \wedge \neg P_{R\downarrow\overline{P}}
\end{aligned}
$$

This means that we don't need to do the final conjuction with $\mathrm{dom}(R)$ when we compute a preimage if we are going to immediately be using the preimage to compute a strong preimage.

This represents an imporant speedup since we will be doing many strong preimage computations where computing $P_{R\downarrow\overline{P}}$ is easy (such as when $P = t_i$) but conjucting it with $\mathrm{dom}(R)$ takes some time since $\mathrm{dom}(R)$ can be complex.

8. Checking Properties under Environmental Constraints

Environmental constraints are constraints on the inputs to the circuit. They are of the form $P[C, T]$ to denote that we only need a constraint to hold if $P[C, T]$ is true. We call such a constraint a "care predicate".

8.1. Index Over Care Predicate Cases via Indexing Relation Restriction

Notice that one easy way to include environmental constraints is to simply add them to the guards of the output constraint. We can then check for the property holding under the enviromental constraint as described above.

However, this can lead to problems for abstractions that merge cases covering $(C_1, T_1), (C_2, T_2)$ where $C_1, T_1 \vDash P$ but $C_2, T_2 \nvDash P$[4]. Specifically, this is more likely to have weak disagreements as these cases cannot assume the environmental constraint holds.

This is referenced to in the 2002 paper where it is recommended to find indexing relations that exactly index cases in P and not any in $\neg P$.

To get such an indexing relation, one straightforward idea is to have a new indexing relation that is the conjuction of the original indexing relation and the input constraint. This will ensure that the indexing relation only ever indexes target variable assignments satisfying the care predicate. However, this can still lead to weak disagreements from the indexing relation being too coarse.

Another way to mitigate the weak disagreements is to use the alternative checking method involving reversing the indexing, described earlier. This can avoid weak disagreements in some cases but can be more computationally expensive.

Given an indexing relation that only indexes the cases in P, we can just apply the symbolic indexing transformation on the antecedent, run the simulation and check the output constraints as described above.

8.1.1. Variation: Condition the Antecedent

We note that doing the combining the above approach with the automatic abstraction would destroy the partitioned abstraction preimage structure, meaning we cannot directly apply the efficient preimage computations described above.

However, we can employ a similar strategy where we first modify each BDD expression E in the antecedent to the form $E' := (E \wedge P) \vee \overline{P} \equiv P \rightarrow E$. We can then do the strong preimage computations, allowing us to only consider the indexing cases we know e to be true when the care predicate is true (corresponding to those cases that map exclusively to the bottom left 3 quadrants in the below image). We modify the guard of the output constraint to include P and do the checking as described above.

Note that compared to when we don't have input constraints, doing this will potentially cause some signals to be $\top$, but only when the indexing variable assignment indexes into only $\overline{P}$. This is actually alright, since P_R not will contain such cases, thus not affecting our output checking procedure (for

[4]Notably, if we don't use abstraction then this should work smoothly. Probably what Jasper Gold does with its recipies.

either correctness or counter example analysis). Since the only indexing cases that we consider during analysis are those that index at least one case in P, this is equivalent to to restriction method described above.

Since we are not modify the indexing relation, we can still use the partitioned abstraction preimage operations which are more efficient.

8.2. Parametric Encoding

An alternative approach would be to use a parametric encoding of the input constraints. A parametric encoding involvings using the `param` function to compute a substition of the input signals with new parameterisation variables.

`param` takes a list of input constraints and a list of signals $s_1, s_2, ..., s_n$ and computes a vector of boolean functions $f_1, f_2, ..., f_n$ from new parameterisation variables $P = \{p_1, ..., p_k\}$ where $k \leq n$ for the purpose of substituting $s_i := f_i(P)$. These functions satisfy the following two conditions:

- (soundness): $\forall P, \langle s_i := f_{i(P)} \mid i \in 1..n \rangle$ satifies the input constraints
- (completeness): $\forall \langle s_1, s_2, ..., s_n \rangle$ that satisfy the input constraints, there exists some P such that $s_i = f_i(P)$

There are two ways to apply this for symbolic simulation.

8.2.1. Parametrise the Indexing Relation

The first stategy is to apply the parametric encoding to transform an independently computed indexing relation. This is described in the 2002 paper. Given a circuit, antecedent, input and output constraints, we will

- Compute `param` on the input constraints
- Substitute each BDD variable in the antecedent with the corresponding function from the parametric encoding
- Compute an indexing relation using the automatic abstraction algorithm
- Substitute each BDD variable in the automatic abstraction result with the corresponding function from the parametric encoding
- There is no need to substitute anything in the output constraint because the guards of the output constraint are just `true`
- Do the indexing transformation on the parameterised antecedent, and output constraints
- Do the symbolic simulation and check whether the output constraint holds

This is a sound approach.

First we note that after we parameterise the antecedent, we still test all the cases C, T such that $P[C, T]$ holds by the completeness of `param`.

We also note that the base case for our symbolic simulation invariant still holds even though our input signals are now functions of the parameterisation variables. In this case, we can imagine that we are doing symbolic simulation on a bigger circuit with the param functions bolted onto the front, feeding the inputs of the regular circuit. The input signals s are now replaced with the param functions f_s and our basecase argument will still hold.

Furthermore, we also have the parameterised indexing relation satisfying the coverage condition:

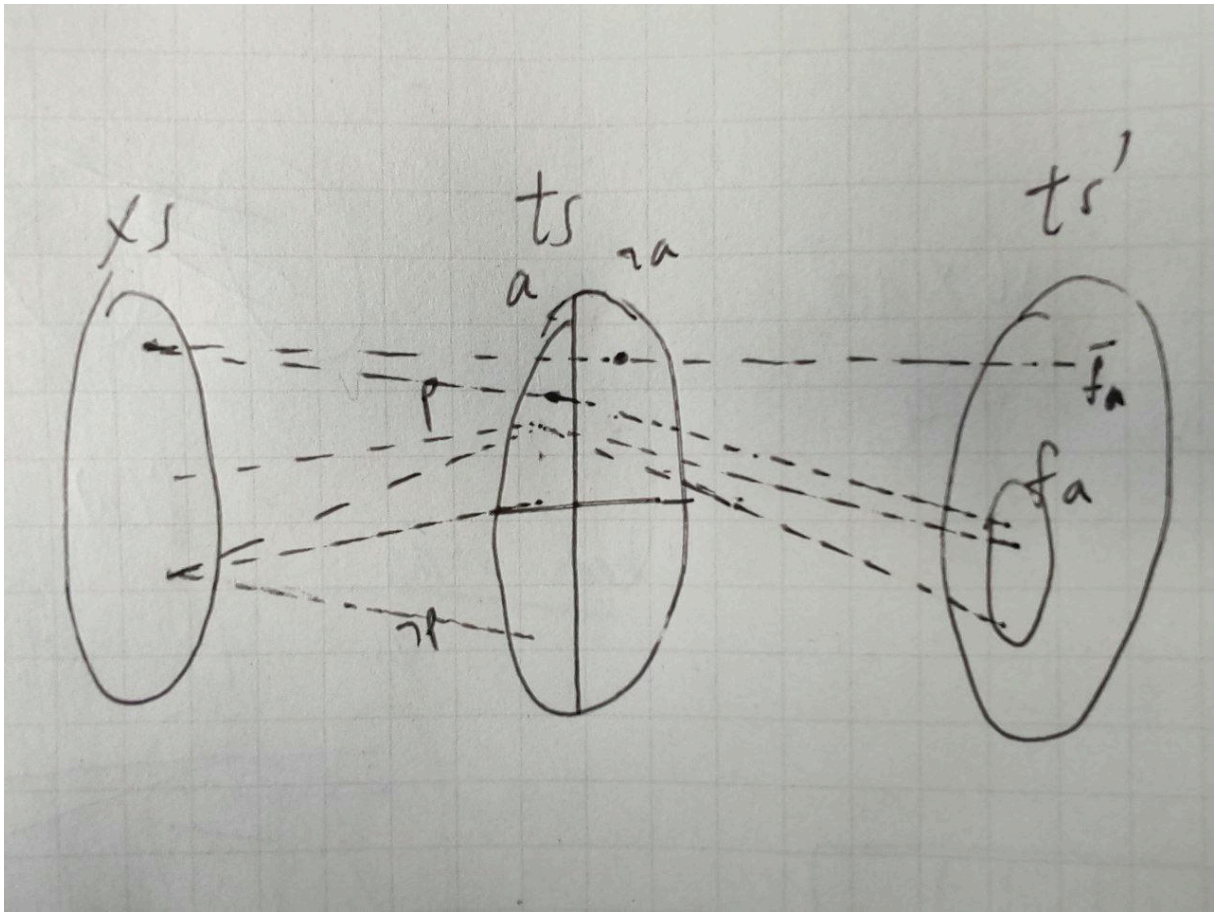
Assuming that the indexing relation $R[X, C, T]$ used satisfies the coverage condition $\forall T \forall C(P[C, T] \rightarrow \exists X R[X, C, T])$ then the parameterised indexing relation $R'[X, C, T']$ will also satisfy the coverage condition, in terms of the new parameterisation variables, i.e. $\forall T' \forall C \exists X R'[X, C, T']$ where T' is the new parameterised input signals.

Proof: Consider an arbitary but fixed C, T'. Let $T = f(T')$ where f is the parametric encoding. We know that $P[C, T]$ is true due to the soundness of `param`. Now, by the fact that $\forall T \forall C(P[C, T] \rightarrow \exists X R[X, C, T])$, we have that $\exists X R[X, C, T]$. Let such $X*$ be such that $R[X*, C, T]$. We observe that $R'[X*, C, T']$ will also hold since we have that $T = f(T')$ and $R' = R[T/f(T')]$.

While this is sound, there are cases where doing the above will lead to a weak disagreement where the property is not proven but there are no counter examples.

Furthermore, this has the problem where the partitioned indexing relation structure is destroyed during the folding of the param substitutions into the indexing relation. This means that we cannot use the partitioned abstraction preimage to compute the preimage of a guard. This makes it computationally infeasible to compute the preimages of the antecedents

8.2.1.1. Relationship to Indexing Restriction Methods



With careful observation, we note that the indexing cases that are included in the strong preimage operation on the antecedent are actually the same whether we are using the above parametric encoding and substitution method or the previously descibred method of modifying the indexing relation to by conujucting it with the input constraint. In either case, we only consider the indexing cases that at least map to one target variable assignment that satisfies both the input constraint and the antecedent bdd variable, and doesn't index any cases that satisfy the input constraint and the logical not of the antecedent bdd variable.

With further observation, the domain of the parameterised indexing relation is actually going to be the same as as the weak preimage of the input constraint under the original indexing relation. Both of them are going to the set of cases that index into at least one case that satisfies the input constraint. As such the output checks will be the same for both methods.

This means that both methods are actually equivalent, i.e. will produce the same results (proven, disproven or unproven).

Since the first method present is equivalent to the more efficient variation, this parametric method is also equivalent to that. Given the better efficiency of the method of conditioning the antecedent, that is preferable in practice.

8.2.2. Parameterise Circuit then Perform Abstraction

While the first strategy of using `param` is not any more effective than the restriction methods, our second strategy is likely to be more effective.

We will do the parameterisation first and then do the automatic abstraction on the circuit with the param functions "bolted" onto the front of the circuit as if the original circuit was just in terms of the parameterisation variables. This will compute a new indexing relation that is not easily found as a modification of the non-parameterised indexing relation like we did in the first method(s).

From here, since the automatic abstraction algorithm gives us coverage by construction, our new indexing relation will satisfy the coverage condition. It is thus sound to use it for symbolic indexing. We modify the antecedent with the parameterisation functions substituted for the original signals and then transform them via the strong preimage under the indexing relation. We then run the symbolic simulation and then just check the output constraint without any guards. Soundness comes from this effectively being symbolic simulation on a bigger circuit, with the modified base case argument still holding.

The soundness of the above procedure can be seen mostly as the same as the regular symbolic simulation correctness.

However, that would make it difficult to use the symbolic constants effectively since we would need to specify that in terms of the new input signals from the param substitution. If the input constraints don't involve the signals that were meant to be the symbolic constants, Jasper Gold allows us to apply param on the other input signals and leave the symbolic constants as is.

8.3. Conditioned Output Constraints

A last way we can deal with input constraints is to just build them into the SystemVerilog Assertions directly as conditions on the input signals to the circuit. This means that we don't need to do anything special at all when we run the symbolic simulation. However, if we are using the input constraints to case split on the possible inputs, we will need to programmtically modify the SystemVerilog Assertions since it would be impractical to do that manually for each environmental constraint

9. Examples

9.1. Content-Addressable Memory (CAM)

We illustrate the effectiveness of symbolic simulation with symbolic indexing on a stripped down version of a CAM.

The CAM stores a fixed number of entries N, each being an integer of length D bits. The CAM also takes in a query of length D bits on each clock cycle and will output on a boolean wire whether the query matches with any of its entries. This wire is called the `hit` since it is high if there is a hit and low otherwise.

On each cycle our CAM will compare the query with each of the entries in parallel and take a logical OR of all the comparisons to get the `hit` wire. The specification of our CAM does a similar operations,

except it compares each entry with the query in sequence, doing many logical ORs in series to check for a hit.

Our property is then whether the `hit` wire of the CAM matches whether the specification found the query in its entries.

9.1.1. Manual Indexing Relation

We construct a partitioned indexing relation manually that correctly provides symbolic indexing for the CAM to be verified via symbolic simulation.

The indexing relation encodes all the different ways that `hit` can be high or low. We will use the following indexing variables to effectively do case splitting:

- h: Boolean variable
  - Whether or not the `hit` wire should be high in the case being considered
- $\mathbf{ch}$: Vector of $\log_2 N$ boolean variables
  - $\mathbf{ch}$ is interpreted as an integer which encodes which entry in the CAM should match with the query in the case being considered, only relevant if h is high.
- $\mathbf{EM}$: Matrix of size N by $\log_2 D$ boolean variables
  - Each $\mathbf{em}_i : \mathrm{em}_i[0], \mathrm{em}_i[1] ..., \mathrm{em}_i[\log_2 D - 1]$ is interpreted as an integer to represent the bit of the ith entry of the CAM that should be different from the query, only relevant if h is low.

We also have the query $\boldsymbol{q}$ as a vector of symbolic constants, with length D.

Based on the descriptions of how the variables work, it is fairly easy to see how we can construct a partitioned abstraction indexing relation. We will add the following conjucts together to get the indexing relation:

A conjuct that covers cases where the `hit` should be high:

$$\forall i \forall j : h \wedge (\mathbf{ch} = i) \Rightarrow (\mathbf{mem}[i][j] = \boldsymbol{q}[j])$$

A conjuct that covers cases where the `hit` should be low:

$$\forall i \forall j : \overline{h} \wedge (\mathbf{em}_i = j) \Rightarrow (\mathbf{mem}[i][j] \neq \boldsymbol{q}[j])$$

Where $0 \leq i < N$ and $0 \leq j < D$.

Equality between boolean vector encoded variables and the loop variables i or j are represented by a big conjuction of the individual bits or their negations to ensure the bit pattern formed from the whole vector matches that of the integer being compared to.

Equality between target variables t and indexing/symbolic constants x is done by adding 2 tuple of the form

- `(t, (premise AND x), false)`
- `(NOT t, false, (premise AND NOT x))`

to the indexing relation. This means that if the premise (such as $h \wedge (\mathbf{ch} = i)$) holds, then t will need to take the value of x. We can create analagous tuples for inequalities between target variables and indexing variables.

Observe that coverage will be achieved since for any given values of the query and the entries of the CAM $\mathbf{mem}$, we will be able to find appropriate values for $h, \mathbf{ch}, \mathbf{em}$ that will satisfy the indexing relation.

Observe that this approach has $1 + \log_2 N + N \log_2 D + D$ indexing variables and symbolic constants. This is exponentially less than the ND variables that are considered without symbolic indexing.

Considering this also lets us know that we should see maximum gain in efficiency of proving wider entry CAMs rather than CAMs with more entries.

9.2. Maximum Circuit

We also showcase the method on a maximum circuit. The maximum circuit takes N inputs, each of length D, and every clock cycle, will output the maximum of all the inputs. This is implemented as a binary tree of 2-input maximum operations between tree nodes, that eventually leads to the output at the root of the tree.

Rather than using a specification that does the same computation in a different way, this circuit has a very natural relational specification that consists of two properties:

- Containment Property: The output of the circuit is one of the input values
- Bounding Property: The output of the circuit is at least as large as all the input values

9.2.1. Manual Indexing Relation

Just like the CAM, we can construct a partitioned indexing relation that will exponentially reduce the number of BDD variables needed for symbolic simulation.

We have the following indexing variables:

- $\boldsymbol{t}$: Vector of D boolean variables
  - $\boldsymbol{t}$ represents the target output of the maximum circuit. I.e. all cases that lead to the circuit producing x will be covered by some indexing cases where $\boldsymbol{t} = x$
- $\boldsymbol{d}$: Matrix of size N by $\log_2 D + 1$ boolean variables
  - $\boldsymbol{d}_i : \boldsymbol{d}_i[0], ..., \boldsymbol{d}_i[\log_2 D]$ encodes an integer that represents the number of most significant bits of the ith input will be the same as the corresponding bits of $\boldsymbol{t}$ before the critical bit of i which will be low in $\mathbf{ins}[i]$ but high in $\boldsymbol{t}$.

We first write our required conjucts in a manner that is easy to reason about, then convert them into a form that we can efficient encode as a partitioned indexing relation. Similarly, we will be having a few conjucts that we will merge together:

A conjuct that ensures that at least one entry will completely match the target output:

$$\bigvee_{i=0}^{n-1} (\boldsymbol{d}_i = D)$$

A conjuct that ensures the most significant bits of each input match with the target output:

$$\forall i : \bigwedge_{j=D-\boldsymbol{d}_i}^{D-1} (\mathbf{ins}[i][j] = t[j])$$

A conjuct to ensure the critical bit for each entry is high in the target output but low in the input:

$$\forall i : \boldsymbol{d}_i < D \Rightarrow (\mathbf{ins}[i][D - \boldsymbol{d}[i] - 1] = 0) \wedge (\boldsymbol{t}[D - \boldsymbol{d}[i] - 1] = 1)$$

Where $0 \leq i < N$.

To actually form the partitioned indexing relation, we need to consider exactly the cases that we wish to force specific target variables to be high or low. This entail rewriting the second and third conjuct.

By rearranging $D - \boldsymbol{d}_i \leq j$ to get $D - j \leq \boldsymbol{d}_i$, we can rewrite the second conjuct as:

$$\forall i \forall j : (D - j \leq \boldsymbol{d}_i) \Rightarrow (\mathbf{ins}[i][j] = \boldsymbol{t}[j])$$

Where $0 \leq j < D$.

By introducing j, we can also rewrite the third conjuct as:

$$\forall i \forall j : (D - 1 - j = \boldsymbol{d}_i) \Rightarrow (\mathbf{ins}[i][j] = 0) \wedge (t[j] = 1)$$

Constructing the actual partitioned indexing relation from the above rules follows a similar process to that done for the CAM. The only new formula is $\boldsymbol{y} \geq x$ for some boolean encoded variable y and a loop variable x. To construct such a formula, we consider binary representation of x and use the following structural recurence:

$$\text{geq}((\boldsymbol{y}_0, ..., \boldsymbol{y}_{n-1}), (x_0, ..., x_{n-1})) \coloneqq (\boldsymbol{y}_{n-1} > x_{n-1}) \vee \text{geq}((\boldsymbol{y}_0, ..., \boldsymbol{y}_{n-2}), (x_0, ..., x_{n-2}))$$
$$\text{geq}((\boldsymbol{y}_0), (x_0)) \coloneqq (\boldsymbol{y}_0 = 1) \vee (x_0 = 0)$$

This takes $D + N \log_2 D$ variables, exponentially less than the ND variables that are considered for symbolic simulation without symbolic indexing.

Furthernote that only the 2nd and 3rd conjucts mention target variables. The first conjuct actually serves as a means of restricting the domain of our indexing relation ones where at least one $\boldsymbol{d}_i$ is equal to D. Compared to the indexing of the CAM, the domain in this case is not just “True”.